Richer State Models

The state model is the type lattice — and it was built two phases ago

2026-08-29

The one-sentence thesis

The system-model tier proves behavioural properties of a process by modelling its state as **one integer**. Everything you'd want next — multiple variables, records, collections, sum-typed protocols, and invariants **discovered rather than annotated** — is not a second research programme. It is a **wiring** job between two things that already exist and were built to fit: the **tag lattice** from the typing phase (which already infers the shape of every value) and the **guarantee engine** from the system phase (which already proves over SMT). This document shows the seam, proves the two load-bearing claims with runnable spikes, and lists what to build.

> **THE THROUGH-LINE**
> The typing phase inferred **shapes**. The system phase proves **behaviour over shapes**. "Richer state models" is the missing wire between them — **type-directed SMT translation**. The information is not missing; it is inferred and sitting one namespace away.

What "state" is today, exactly

Read `priv/system/core.bl`. `verify-process` collects the state variables an invariant mentions, and then:

```clojure
svars (state-vars inv-node clauses)
svar  (if (empty? svars) nil (first (into [] svars)))   ; ← takes the FIRST
```

And `priv/system/smt.bl` states its own ceiling honestly:

> "State vars are modelled as SMT integers — the common case for process state (balances, counters, phases-as-ints). Non-integer state (maps, strings) is out of this translator's scope and surfaces as `:untranslatable`."

So the model of a process's state is: **one integer that moves**. `balance = 5`, guarded by integer arithmetic, discharged by z3. When state is anything else, the translator returns `:untranslatable` and the checker stays silent rather than lie. That is a real and sound floor — and it is a floor, not a ceiling, only because the richer information is being thrown away, not because it is unavailable.

Claim 1 — most of the work is already done

The lift to richer state has two halves: **infer the structure** and **prove over it**. The first half is the entire typing phase, already shipped. The second half is the guarantee engine, already shipped. What is missing is the adaptor between them. Here is the inventory.

capability	already built, where	status
value shapes	`typed/all-tags` lattice: `:int :float :map :vec :set :string :bool :fn :nil :sym :seq :list` — every value's tag inferred by `typed/walk`	✓ shipped
flow narrowing	`walk-if` / `guard-narrow`: `(if (int? x) …)` refines `x` to `:int` in the then-branch, `:diff` in the else — path-sensitive	✓ shipped
record fields	`bind-pattern` destructures `{:keys [...]}` and map patterns, binding each field	✓ shipped
signatures from source	`sigs-from-env` reads `^{:args :ret}` off name meta through the real compile pipe — no re-declaration	✓ shipped
multi-var SMT	`preserves?` already declares `svar` + `svar2` + inputs and primes; N is a loop where there is a `first`	◐ one line away
candidate predicates	`system.core/candidates` — miniKanren enumerates `(op a b)` over a domain, deduped	✓ shipped
z3 preservation	`preserves?` / `sufficient?` / `establishes?` — the Hoare triple discharge	✓ shipped
effect / footprint	`system.footprint/fp-walk` reads resource modes from a body — never annotated	✓ shipped

The only ◐ in the table is Rung 1, and it is genuinely a truncation bug: `state-vars` **collects the set** of state variables, and `verify-process` **keeps one**. The z3 layer beneath it (`preserves?`) already handles a current var, a primed var, and declared inputs — it was always N-ary; only the seam narrows it to one.

Proof: multi-variable relational invariants (Rung 1)

To show the machinery generalizes, `research/p18_state_models/spike_multivar.bl` models a two-variable state with a **relational** invariant — one that a single-integer model **cannot express**:

```clojure
state      {:balance b, :reserved r}
invariant  (and (>= reserved 0) (>= balance reserved))
           ;; reserved funds are non-negative AND never exceed the balance
```

The check declares the whole tuple, primes it jointly (`balance2` , `reserved2`), and asks z3 whether any transition can break the coupled predicate.

```
$ elixir … research/p18_state_models/spike_multivar.bl
```

```
reserve  preserves invariant: true
release  preserves invariant: true
withdraw preserves invariant: true   (guards against FREE funds)
buggy withdraw preserves invariant: false   (guards whole balance)
PASS: three good transitions proven, the reservation-ignoring bug caught
```

> The relational invariant `balance ≥ reserved` is proven for three transitions, and a buggy `withdraw` that guards the whole balance (ignoring the reservation) is caught with the exact reasoning a single-variable model is blind to: it can drive `balance < reserved` . The z3 mechanism needed **no change** — only the seam that today writes `(first svars)` needs to declare and prime the whole set.

Claim 2 — the way it was built leads to crazier things

Here is the part worth slowing down for. The typing phase did not build a type **checker**. It built a **relation** and a set of **engines**, and a relation runs in every direction. That design choice — visible in `typeo` and in the strict engine division — means the richer-state ladder does not just climb to "more types". It climbs to capabilities that were never on the original list.

The relation runs backwards: synthesis, not just checking

`examples/typing/04_holes_demo.bl` defines `typeo` , a bidirectional typing relation in miniKanren. The demo's own words:

> "The relation is the same artifact that checks (forwards), refutes (sideways), and synthesizes (backwards): `typeo` ."

A checker answers "does this term have this type?". A **relation** also answers "**what terms have this type?**" — run `typeo` with the term as the unknown and it **enumerates inhabitants**. The holes demo fills a typed hole with real expressions from the local context; the ill-typed hole gets no answer.

Now apply that to **state**. Once state has a type richer than `Int` , "what transition preserves this invariant?" is the **same backward run**. The tier stops being "prove the invariant the human wrote" and becomes "**synthesize the guard that makes the invariant hold**" — program repair, from the same relation, for free.

The engines were kept strictly non-overlapping — so they compose

The system phase inherited a discipline from the typing phase: **four engines, no fallbacks, strict ownership**.

> **tag lattice** owns structure · **z3** owns arithmetic decisions · **miniKanren** owns relations and synthesis · **datalog** owns the codebase-as-facts. Effects are a fifth, folded in as a footprint lattice.
> **Because** — each engine answers a different KIND of question, so a richer state model recruits all four without any one leaking into another's territory

That non-overlap is why the ladder is composable rather than a rewrite. A sum-typed protocol state, at the summit, uses **all four at once**: the lattice gives the variant tags, z3 proves the per-variant arithmetic, miniKanren synthesizes the missing transition, datalog checks every **caller** speaks the protocol. None of them has to learn another's job.

The crazier thing, proven: invariants discovered from nothing

The boldest consequence. Today you must **write** `^{:invariant (>= balance 0)}`. But the machinery to **discover** it is already present: `candidates` enumerates predicates, `preserves?` decides them, and the state vars come from the lattice. Assemble those into a **Houdini fixpoint** (Flanagan–Leino) — start from every candidate that holds at the initial state, drop any conjunct a transition can break, iterate — and the invariant **falls out of the code**.

`research/p18_state_models/spike_discover.bl` runs exactly this on an **unannotated** account machine:

```
$ elixir … research/p18_state_models/spike_discover.bl
```

```
candidate vocabulary (6): (<= balance 0) (<= balance 1) (<= balance 5)
                          (>= balance 0) (>= balance 1) (>= balance 5)
hold at init (balance=0): (<= balance 0) (<= balance 1) (<= balance 5) (>= balance 0)
∧ survive induction:      (>= balance 0)
DISCOVERED INVARIANT:  (and (>= balance 0))
PASS: discovered (>= balance 0) from nothing; refuted the bogus candidates
```

> From an account with **no annotation**, the loop discovers `(>= balance 0)` and refutes every bogus candidate (`<= balance 0`, `>= balance 5`, …). The one subtlety, pinned in the spike: Houdini must be seeded with a **consistent** set, so the establishment filter (what holds at `init`) runs **before** induction — otherwise the contradictory candidates make the antecedent unsatisfiable and induction is vacuous. Order matters; the result is real.

This is `abduce`'s existing search aimed at the **whole** invariant instead of a missing conjunct, seeded by the inferred state vars. "Implied invariants" was never a missing engine — it was a loop we had not yet written around engines we already had.

The ladder

Each rung is a concrete capability, and each names the existing machinery it recruits. The difficulty is honest: the top two rungs cross z3's decidability line and pay for it in the **approximate-guarantee catalog** the system tier already ships (`exact-guarantees` vs `approximate-guarantees`).

rung	state model → what it buys	lift
1 · multi-var	tuple of ints; relational invariants (`balance ≥ reserved`). Rate limiters, pools, any coupled counters. **Shipped** — live in `verify-process`, demo 10, tests green.	small

rung	state model → what it buys	lift
2 · records	product of typed fields (int + bool + …); mode/flag machines (`frozen ⇒ balance = 0`). Projection: each field its own z3 sort — the lattice already knows the sorts	medium
3 · collections	z3 arrays / sequences / sets; queue bounds, no-duplicate, mailbox growth. Crosses into **bounded** guarantees — the catalog already distinguishes them	high
4 · sum types	tagged variants (`:idle | (:running job) | :done`); legal-transition protocols, session types. Recruits all four engines; connects to the existing `simulates?` / verified-hot-upgrade machinery	high
★ · discovery	invariant synthesis over any rung's vocabulary via Houdini. No annotation. **Proven above** at Rung 1′s vocabulary	medium

> **TYPE-DIRECTED TRANSLATION IS THE WHOLE MECHANISM**
> Today `smt.bl` hardcodes `Int` . The lift is: for each state var, ask the tag lattice its type and emit the matching z3 sort — `Int` , `Bool` , `(Seq …)` , `(Array …)` , a declared datatype. `state ≜ ⟦type-lattice⟧` . The translator stops assuming and starts **consulting**. That single indirection is Rungs 2–4.

What is already inferred (the "implied everything" audit)

The user asked specifically what is **implied** rather than annotated. The answer is: nearly everything except the top-level invariant, and even that is now in reach.

property	inferred?	where
types	✓	the entire tag-lattice phase — no annotation needed
effects / footprints	✓	`fp-walk` reads resource modes from the body
process name (when unnamed)	✓	`model/content-hash` — `υ⟨labels⟩·hash` over the transition graph
handled message protocol	✓	`handled-labels` reads the accepted set from the clauses
unhandled messages	✓	join of `:call/*` senders against handled — the check TLA$^+$ cannot make
missing preconditions	✓	`abduce` — miniKanren enumerates, z3 decides, shortest-first

property	inferred?	where
return types	✓	`codebase/infer-clause-ret` — `typed/walk` over the body when unannotated
termination measures	◐	`termination` proves `dec` / `rest` shrinking; `^{:decreasing}` is the escape hatch
the top-level invariant	★	**was** annotate-only; the discovery spike above closes it

> `abduce` **strengthens** a given invariant with a missing conjunct; `all-senders-guarantee?` **infers a whole-program obligation** from the call graph. Discovery is those same engines run as a fixpoint over the lattice's state vars — the last "annotate-only" property becomes inferred.
> **Because** — the two auto-strengthening pieces already ship, and discovery is the same search with a wider target

The build plan, and the demos it produces

The work is filed as `PLAN-050`. Six pieces, dependency-ordered, each shipping as **tests and a demo** (the tier's graduation contract). The demos are the proof surface — here is what will exist when the plan is done:

demo	the story it tells
`examples/system/10_multivar_invariant.bl`	a reservation account: `balance ≥ reserved ≥ 0` proven; the reservation-ignoring withdraw caught — a bug invisible to single-variable models
`examples/system/11_record_state.bl`	a mode machine: `frozen ⇒ balance = 0` over a `{:balance int :frozen bool}` record; a transition that pays out while frozen caught — mixed-sort state
`examples/system/12_collection_state.bl`	a bounded work queue: "the pending queue never exceeds capacity" as a **bounded** guarantee, registered honestly in the approximate catalog
`examples/system/13_protocol_state.bl`	a connection with `:idle | :open | :closed`; an illegal `:send`-while-`:closed` transition caught — sum-typed protocol conformance

demo	the story it tells
`examples/system/14_discover_invariant.bl`	point the checker at an **un-annotated** server; it **prints the invariant it discovered** and then proves it — the headline
`examples/system/15_type_directed.bl`	the same server, three state shapes (int, record, sum); the **one** translator adapts by consulting the lattice — showing the mechanism, not just the results

Where it stands

Rung 1 is no longer a spike — it is **shipped**: `verify-process` now handles N state variables jointly, the reservation account with its relational invariant `balance ≥ reserved` proves end-to-end through the real checker, the reservation-ignoring withdraw is caught, and the single-variable seam is unchanged (N=1 is one param in the `define-fun`). `examples/system/10_multivar_invariant.bl` demonstrates it; the seam tests gain two cases; the non-auth suite stays green at 637 tests / 1736 assertions / 0 failures.

The second claim — invariants **discovered from nothing** via a Houdini fixpoint over the existing abduce vocabulary — is proven in `spike_discover` and filed as `p18-discover` for graduation. The lift to Rungs 2–4 is type-directed SMT translation: consulting the tag lattice for each state var's sort instead of assuming `Int`. Nothing here is a new engine; all of it is a wire between two phases that were, in retrospect, built to be connected. That is the sense in which most of the work was already done — and the sense in which the way it was done reaches further than the original proposal: a relation that synthesizes, four engines that compose, and an invariant that no longer needs a human to state it.